

-2 / +19 lines of 13 · 2 parallel region(s) added · 6 of 11 candidate(s) kept · RESTRUCTURING (15 code line(s) added, 2 removed) · restructuring level 0

BEFORE	AFTER	file
1#include "data.h"	1#include "data.h"	
	2#include <stdlib.h>	
2	3	
3real_t kernel_k53(void)	4real_t kernel_k53(void)	
4{	5{	
	6real_t *u_work = (real_t *)malloc(sizeof(real_t) * LEN_1D);	
	7real_t *v_work = (real_t *)malloc(sizeof(real_t) * LEN_1D);	
	8	
5for (int nl = 0; nl < iterations; nl++) {	9for (int nl = 0; nl < iterations; nl++) {	
	10#pragma omp parallel for shared(v_work,u_work)	
	11for (long i = 0; i < LEN_1D; i++) {	
	12u_work[i] = u[i];	
	13v_work[i] = v[i];	
	14}	
6for (long i = 1; i < LEN_1D; i++) {	15for (long i = 1; i < LEN_1D; i++) {	
7u[ju[i]] += v[kv[i]] * c[i];	16u_work[ju[i]] += v_work[kv[i]] * c[i];	
8v[jv[i]] = u[ku[i]] * d[i] + c[i];	17v_work[jv[i]] = u_work[ku[i]] * d[i] + c[i];	
	18}	
	19#pragma omp parallel for shared(v_work,u_work)	
	20for (long i = 0; i < LEN_1D; i++) {	
	21u[i] = u_work[i];	
	22v[i] = v_work[i];	
9}	23}	
10dummy(a, b, c, d, e);	24dummy(a, b, c, d, e);	
11}	25}	
	26	
	27free(u_work);	
	28free(v_work);	
12return (real_t)0;	29return (real_t)0;	
13}	30}	